

main

November 11, 2025

```
[1]: import matplotlib.pyplot as plt
from pathlib import Path
import nevergrad as ng
from tqdm import tqdm
import pandas as pd
import numpy as np
import pickle
import optuna
import plotly

[2]: def load_array_data(base_dirs: dict[str, Path]):
    """
    Load all EDM data as arrays.

    Returns:
        data: dict
            data[edm_state][run_id] = {
                "X": X, # lambda_frac grid
                "Y": Y, # theta grid (mrad)
                "Z": Z, # probability density
            }
    """
    data = {}
    for edm_state, dir_path in base_dirs.items():
        state_data = {}
        for file in sorted(dir_path.glob("G2_*.pkl")):
            run_id = int(file.stem.split("_")[1])
            with open(file, "rb") as f:
                X, Y, Z = pickle.load(f)
            state_data[run_id] = {
                "X": X,
                "Y": Y * 1000,
                "Z": Z,
            }
        data[edm_state] = state_data
    return data
```

```

[3]: def plot_heatmap(X, Y, Z, w=None, title="", xlabel="", ylabel="",
                    cbar_label="", save_path=None):
    """
    Generic 2D heatmap plotter with optional weighting.
    Normalises using  $p = Z * d * d$  so that  $\sum(p_i * w_i_{norm}) = \sum(w_i)$ .
    """
    # Compute bin widths
    dtheta = np.abs(Y[1, 0] - Y[0, 0])
    dlambda = np.abs(X[0, 1] - X[0, 0])
    p = Z * dtheta * dlambda # actual probability per bin

    if w is not None:
        # Compute normalisation constant
        numerator = np.sum(p)
        denominator = np.sum(p * w)
        N = numerator / denominator if denominator > 0 else 1.0
        Z_plot = Z * (N*w if w is not None else 1.0)

    plt.figure(figsize=(6, 5))
    plt.pcolormesh(X, Y, Z_plot, shading='auto', cmap='viridis')
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.colorbar(label=cbar_label)
    plt.tight_layout()

    if save_path is not None:
        plt.savefig(save_path, dpi=150)
        plt.close()
    else:
        plt.show()

def save_probability_maps(data_all, output_base=Path("outputs"), save=True,
    plotting=False):
    if not save:
        print("Skipping save - files already generated.")
        return

    for edm_state, runs in tqdm(data_all.items(), desc="EDM States"):
        folder_name = f"{edm_state} EDM"
        output_dir = output_base / folder_name
        output_dir.mkdir(parents=True, exist_ok=True)

        for run_id, grids in runs.items():
            X, Y, Z = grids["X"], grids["Y"], grids["Z"]

```

```

fname = f"G2_{run_id:02}_{edm_state.lower()}EDM.png"
save_path = output_dir / fname

plot_heatmap(
    X, Y, Z,
    title=f"{folder_name} - Run {run_id:02}",
    xlabel=r"Fractional energy ( $\lambda$ )",
    ylabel=r"Longitudinal angle ( $\theta_L$ ) [mrad]",
    cbar_label="Probability density",
    save_path=save_path
)

```

```

[4]: base_dirs = {
    "Without": Path("data/Without EDM"),
    "With": Path("data/With EDM"),
}

data_all = load_array_data(base_dirs)

save_probability_maps({"With": data_all["With"]},
    ↪output_base=Path("outputs"), save=False)

```

Skipping save - files already generated.

```

[5]: def calculate_metrics_array(Z, Y, N, w=None):
    probs = N * Z * 1e-3

    if w is None:
        n = np.sum(probs)
        if n == 0:
            return np.nan, np.nan, np.nan
        mean = np.sum(probs * Y) / n
        var = np.sum(probs * (Y - mean) ** 2) / n
        sig = np.sqrt(max(var, 0.0))
        sem = sig / np.sqrt(n)
        return mean, sig, sem

    # Weighted version
    w = np.abs(w)
    n = np.sum(probs * w)
    if n == 0:
        return np.nan, np.nan, np.nan

    neff = (np.sum(probs * w) ** 2) / np.sum(probs * w ** 2)
    neff = max(neff, 1.0)

    mean = np.sum(w * probs * Y) / n
    var = np.sum(probs * w * (Y - mean) ** 2) / n

```

```

    sig = np.sqrt(max(var, 0.0))
    sem = sig / np.sqrt(neff)
    return mean, sig, sem

# --- FOM calculator ---
def compute_fom_for_lambda(data_all, n_val, lambda_curr, weight_func=None,
    plotting=False, normalize_weights=True):
    X = data_all["With"][0]["X"][0, :]
    i_lambda = np.argmin(np.abs(X - lambda_curr))

    run_means = []
    run_sems = []

    for run_id, grids in data_all["With"].items():
        Z = grids["Z"][:, i_lambda]
        Y_vals = grids["Y"][:, 0]

        # --- Apply weights properly ---
        if weight_func is not None:
            full_w = weight_func(grids["Y"], grids["X"])

            # Ensure shape alignment
            if full_w.shape != grids["Z"].shape:
                print(f" Shape mismatch for run {run_id}: w={full_w.shape},
    Z={grids['Z'].shape}")
                full_w = np.resize(full_w, grids["Z"].shape)

            w = np.abs(full_w[:, i_lambda])

            if normalize_weights:
                w /= np.nanmax(full_w)

        else:
            w = None

        mean, sig, sem = calculate_metrics_array(Z, Y_vals, n_val, w=w)
        run_means.append(mean)
        run_sems.append(sem)

    means_with = np.array(run_means)
    sems_with = np.array(run_sems)
    valid = np.isfinite(means_with) & np.isfinite(sems_with) & (sems_with > 0)

    FOM = np.sum(means_with[valid] ** 2 / (2 * sems_with[valid] ** 2)) if np.
    any(valid) else np.nan
    return {"lambda_frac": lambda_curr, "FOM": FOM}

```

```

def compute_fom_over_lambda(
    data_all, n_val, lambda_min=0.2, lambda_max=0.95, lam_steps=None,
    weight_func=None, plotting=False, normalize_weights=True
):
    X = data_all["With"][0]["X"][0, :]
    if lam_steps is None:
        min_indx = np.argmin(np.abs(X - lambda_min))
        max_indx = np.argmin(np.abs(X - lambda_max))
        sliced_lam = X[min_indx:max_indx]
    else:
        sliced_lam = np.linspace(lambda_min, lambda_max, lam_steps)

    FOM = [
        compute_fom_for_lambda(
            data_all, n_val, lam_curr,
            weight_func=weight_func, plotting=plotting,
            normalize_weights=normalize_weights
        )["FOM"]
        for lam_curr in sliced_lam
    ]

    FOM = np.array(FOM, dtype=float)
    FOM = FOM[np.isfinite(FOM)]
    return FOM

```

```
[6]: n_vals = [1,10,100,1000,2500,5000,7500,10000]
```

```

for n in n_vals:
    FOM = compute_fom_over_lambda(data_all, n)
    print(f"{n}: {np.sum(FOM)}")

```

```

1: 19.031433057572503
10: 190.31433057572502
100: 1903.1433057572503
1000: 19031.4330575725
2500: 47578.58264393125
5000: 95157.1652878625
7500: 142735.74793179374
10000: 190314.330575725

```

```

[7]: edm_state = "With"
run_id = 7
grids = data_all[edm_state][run_id]
TH, LAM, Z = grids["Y"], grids["X"], grids["Z"]

weight_functions = [
    ("Uniform 1s", lambda Y, X: np.ones_like(Y)),

```

```

    ("Uniform 5s", lambda Y, X: 5 * np.ones_like(Y)),
    ("Central energy", lambda Y, X: np.exp(-((X / 10)**2))),
    ("Edge energy", lambda Y, X: 1 - np.exp(-((X / 10)**2))),
    ("Central theta", lambda Y, X: np.exp(-((Y / 10)**2))),
    ("Edge theta", lambda Y, X: 1 - np.exp(-((Y / 10)**2))),
    ("Small angle, large energy", lambda Y, X: np.exp(-((Y / 100)**2)) * X),
    ("Large angle, small energy", lambda Y, X: (1 - np.exp(-((Y / 100)**2))) * X),
]

plotting = True
if plotting:
    for name, func in weight_functions:
        w = func(TH, LAM)
        plot_heatmap(
            LAM, TH, Z, w=w,
            title=name,
            xlabel=r"$\lambda$ (energy fraction)",
            ylabel=r"$\theta$ (mrad)",
            cbar_label="Weighted prob. density",
            save_path=f"outputs/Weighting/{name}.png"
        )
else:
    print("Plotting disabled.")

```

```

[8]: # Storage for results
metrics = {}

# Loop through each analytic weight
for name, weight_func in weight_functions:
    print(f"Evaluating weighting: {name}")
    # Compute FOM for this weighting
    metrics[name] = compute_fom_over_lambda(
        data_all=data_all,
        n_val=10000,
        lam_steps=10,
        weight_func=weight_func,
        plotting=plotting
    )

```

```

Evaluating weighting: Uniform 1s
Evaluating weighting: Uniform 5s
Evaluating weighting: Central energy
Evaluating weighting: Edge energy
Evaluating weighting: Central theta
Evaluating weighting: Edge theta
Evaluating weighting: Small angle, large energy
Evaluating weighting: Large angle, small energy

```

```
[9]: fig = plt.figure(figsize=(6, 6))

lambdas = np.linspace(0.2,0.95,10)
for name, _ in weight_functions:
    foms = metrics[name]
    print(f"{name}, {max(foms):.3g}")
    plt.plot(lambdas, foms, label=name)

plt.xlabel(r"$\lambda$")
plt.ylabel(r"FOM($\lambda$)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Uniform 1s, 571

Uniform 5s, 571

Central energy, 571

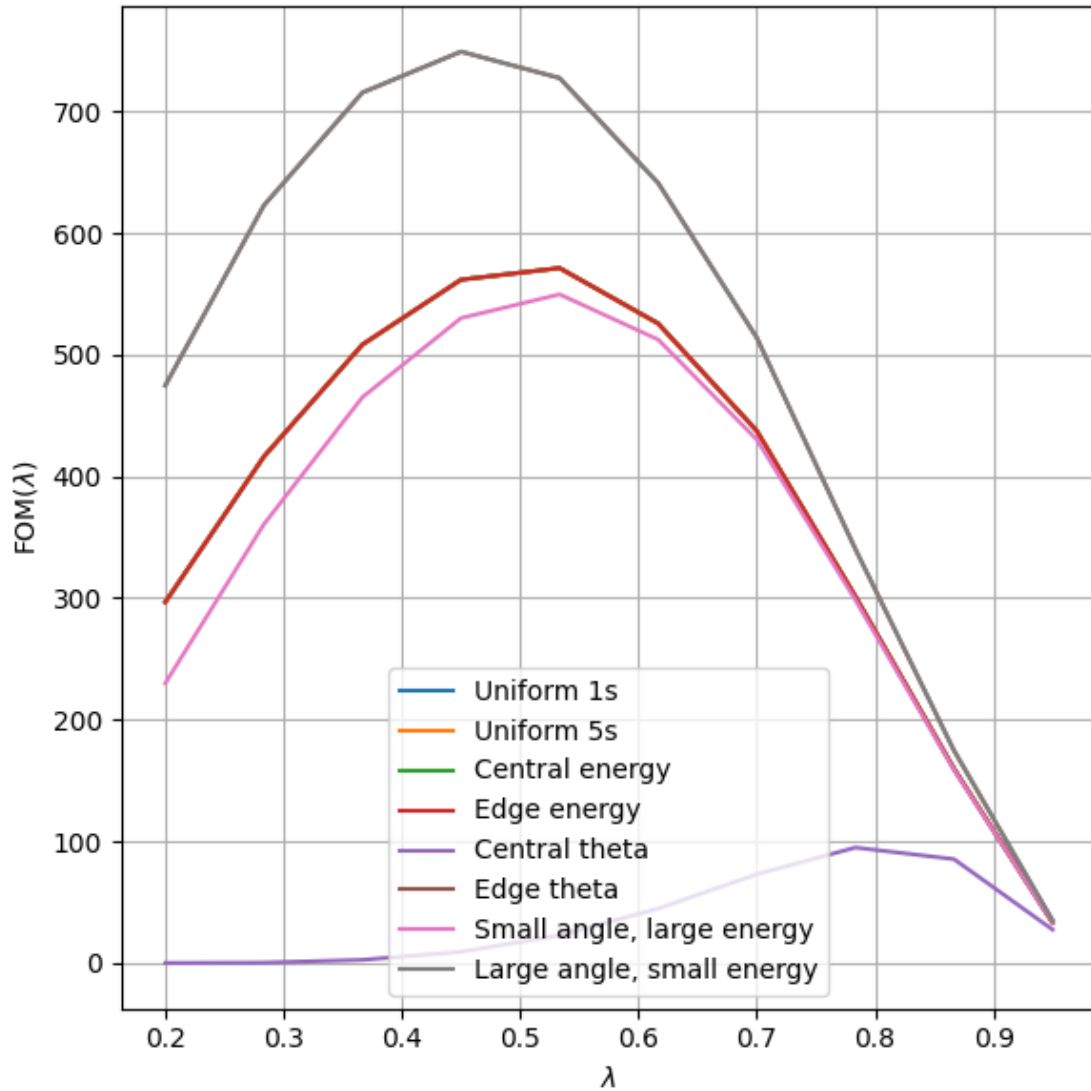
Edge energy, 571

Central theta, 94.9

Edge theta, 749

Small angle, large energy, 550

Large angle, small energy, 749



```
[10]: # --- Parameters ---
plotting = False
output_dir = Path("outputs/Difference")
output_dir.mkdir(parents=True, exist_ok=True)

# --- Loop over all runs ---
for run_id in range(29):
    # Extract "With" and "Without" data for this run
    grids_with = data_all["With"][run_id]
    grids_without = data_all["Without"][run_id]

    Y, X = grids_with["Y"], grids_with["X"]
    Z_with = grids_with["Z"]
```



```

Z_without = grids_without["Z"]

# --- Compute difference (With - Without) ---
Z_diff = Z_with - Z_without

# --- Plotting ---
if plotting:
    plt.figure(figsize=(6, 5))
    plt.pcolormesh(X, Y, Z_diff, shading="auto", cmap="coolwarm")
    plt.xlabel(r"$\lambda$ (energy fraction)")
    plt.ylabel(r"$\theta$ [mrad]")
    plt.title(fr"$\Delta$ Probability Density (With - Without EDM) for Run_
↵= {run_id:02}")
    plt.colorbar(label="Difference")
    plt.tight_layout()
    plt.savefig(output_dir / f"Difference_{run_id:02}.png", dpi=150)
    plt.show()

```

```

[11]: # --- Parameters ---
run_id = 7
plotting = True
output_dir = Path("outputs/Weighting")
output_dir.mkdir(parents=True, exist_ok=True)

# --- Extract base grid and difference ---
grids_with = data_all["With"][run_id]
grids_without = data_all["Without"][run_id]

Y, X = grids_with["Y"], grids_with["X"]
Z_with = grids_with["Z"]
Z_without = grids_without["Z"]

# --- Compute difference map (With - Without) ---
Z_diff = Z_with - Z_without

# --- Use the difference as a weight map ---
w = Z_diff.copy()
name = "Difference"

# --- Apply weighting to base probability density (e.g., "With" EDM) ---
Z_weighted = Z_with * w

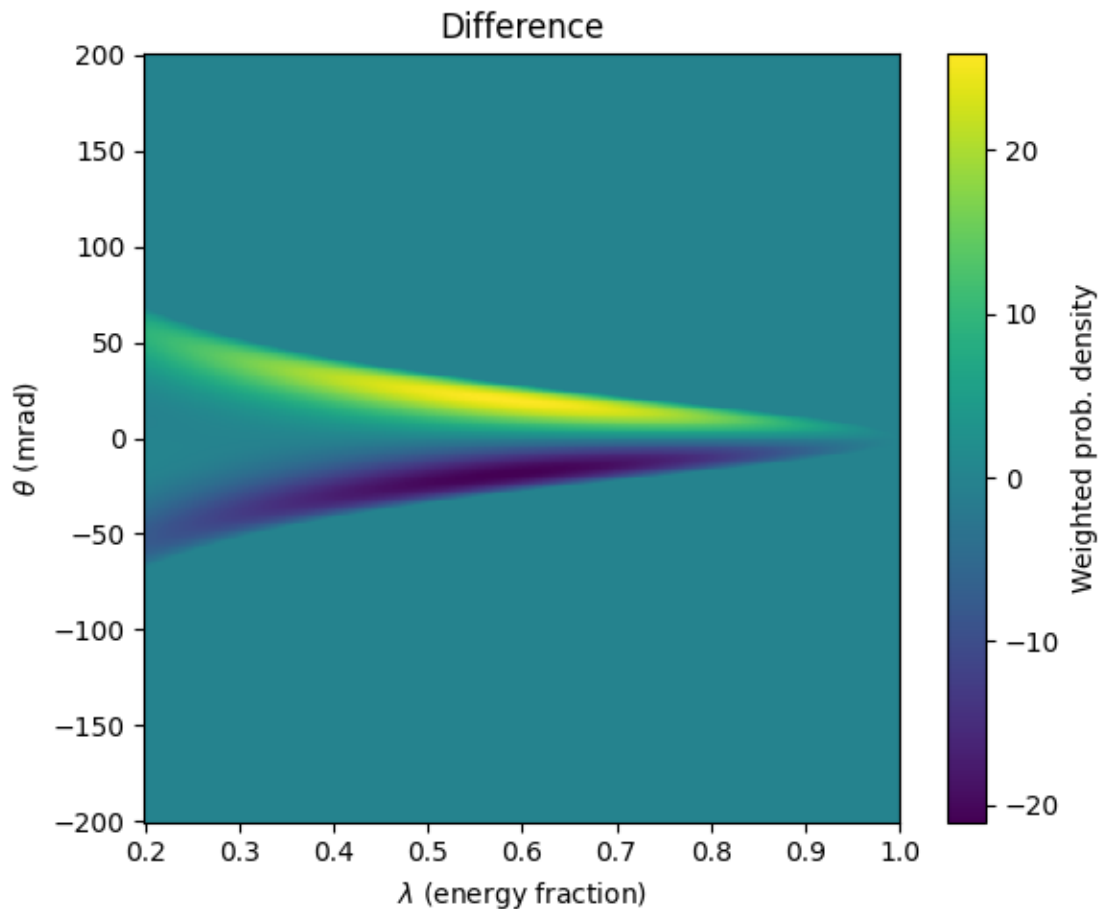
# --- Plot result ---
plt.figure(figsize=(6, 5))
plt.pcolormesh(X, Y, Z_weighted, shading="auto", cmap="viridis")
plt.title(name)
plt.xlabel(r"$\lambda$ (energy fraction)")

```

```

plt.ylabel(r"$\theta$ (mrad)")
plt.colorbar(label="Weighted prob. density")
plt.tight_layout()
plt.savefig(output_dir / f"{name}.png", dpi=150)
plt.show()

```



```

[12]: # --- Uniform weight ---
uniform_weight = lambda Y, X, run_id=None: np.ones_like(Y)
FOM_unweighted = np.sum(compute_fom_over_lambda(
    data_all, 10000, weight_func=uniform_weight
))

# --- Difference weight ---
# Precompute the difference grid from the reference run (run_id = 7)
ref_id = 7
Z_with_ref = data_all["With"][ref_id]["Z"]
Z_without_ref = data_all["Without"][ref_id]["Z"]

```

```

diff_weight_grid = np.abs(Z_with_ref - Z_without_ref)

# Optional: normalize globally (so it's between 0-1)
diff_weight_grid /= np.nanmax(diff_weight_grid)

def diff_weight_func(Y, X, run_id=None):
    """
    Return the precomputed |Z_with - Z_without| grid for run_id=7.
    """
    return diff_weight_grid

FOM_diff = np.sum(compute_fom_over_lambda(
    data_all, 10000, weight_func=diff_weight_func
))

print(
    f"Unweighted={FOM_unweighted:.4f} "
    f"Diff-weighted={FOM_diff:.4f}"
)

```

Unweighted=190314.3306 Diff-weighted=233496.1148

```

[13]: def plot_weight(Y, X, W, name="Weight", save_path=None):
    """
    Plot 1D (vs theta) and 2D contour of a weight array.

    Parameters
    -----
    Y, X : 2D arrays
        Theta (Y) and lambda (X) coordinate grids.
    W : 2D array
        Weight array to visualise.
    name : str
        Label or title for the weight.
    save_path : str or Path
        Optional - if provided, saves instead of showing inline.
    """

    # 1D slice through the central lambda index
    mid_idx = W.shape[1] // 2
    theta_vals = Y[:, 0]
    weight_slice = W[:, mid_idx]

    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    # --- 1D plot ---

```

```

axes[0].plot(theta_vals, weight_slice, color="blue", lw=1.5)
axes[0].set_xlabel(r"$\theta_L$ [rads]")
axes[0].set_ylabel("Weight")

# --- 2D contour plot ---
c = axes[1].pcolormesh(X, Y, W, shading="auto", cmap="Blues")
axes[1].set_xlabel(r"Fractional Lab Frame Energy ($\lambda$)")
axes[1].set_ylabel(r"Longitudinal Angle ($\theta_L$) [rads]")
fig.colorbar(c, ax=axes[1], label="Weighting (w)")

plt.tight_layout()
if save_path is not None:
    plt.savefig(save_path, dpi=200)
    plt.close()
else:
    plt.show()

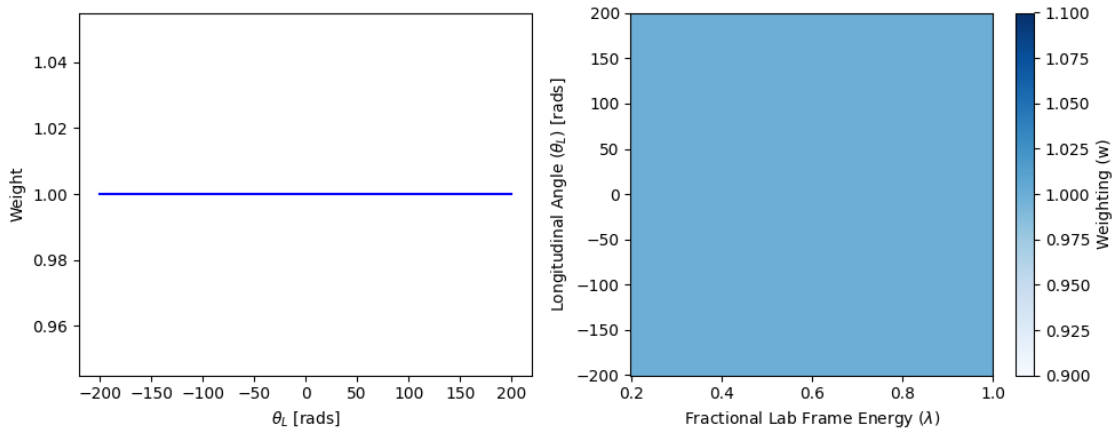
```

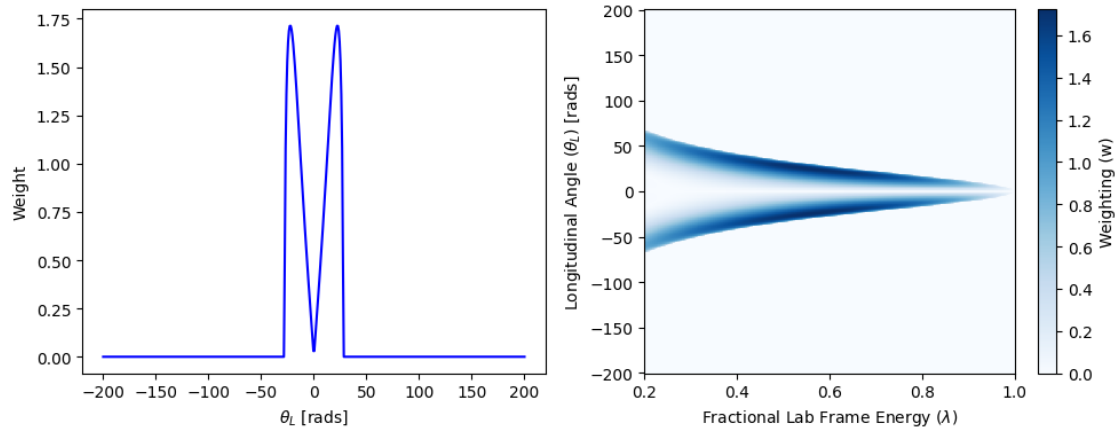
```

[14]: # Example 1: uniform weight
run_id = 7
Y, X, Z = data_all["With"][run_id]["Y"], data_all["With"][run_id]["X"],
↳data_all["With"][run_id]["Z"]
W_uniform = np.ones_like(Z)
plot_weight(Y, X, W_uniform, name="Uniform")#, save_path="outputs/Weighting/
↳uniform 2d plot.png")

# Example 2: difference weight
Z_with = data_all["With"][run_id]["Z"]
Z_without = data_all["Without"][run_id]["Z"]
W_diff = np.abs(Z_with - Z_without)
plot_weight(Y, X, W_diff, name="Difference Weight")#, save_path="outputs/
↳Weighting/difference 2d plot.png")

```



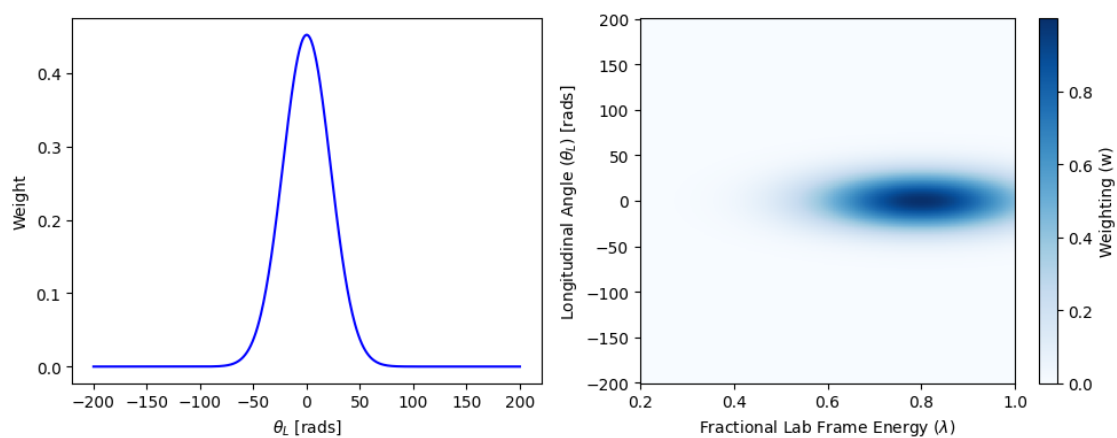
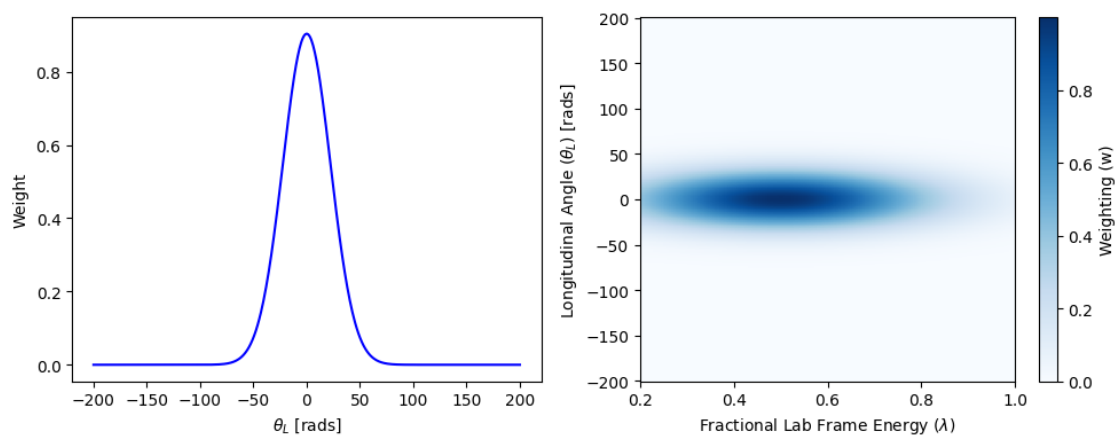
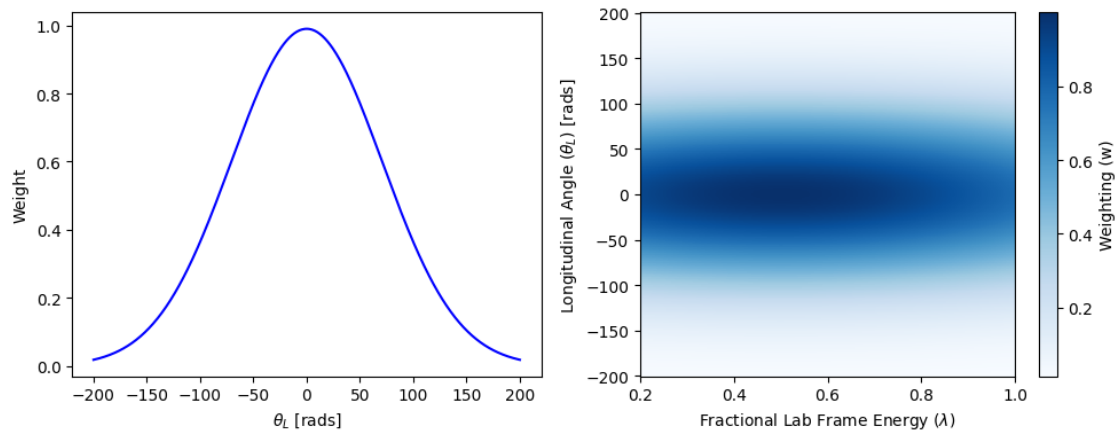


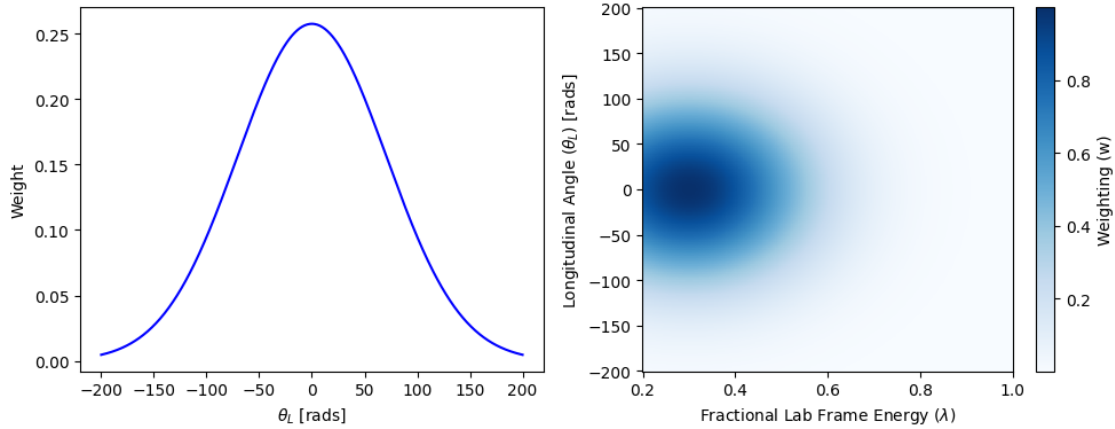
```
[15]: def parametric_weight(Y,X,a_theta,a_lambda,b_lambda):
        """
        Simple parametric weighting function
        """
        return np.exp(-a_theta * Y**2) * np.exp(-a_lambda * (X - b_lambda)**2)

[16]: run_id = 7
Y, X, Z = data_all["With"][run_id]["Y"], data_all["With"][run_id]["X"],
↳data_all["With"][run_id]["Z"]

# Example Gaussian weight parameter sets
example_params = [
    {"a_theta": 1e-4, "a_lambda": 1, "b_lambda": 0.5}, # Broad, almost uniform
    {"a_theta": 1e-3, "a_lambda": 10, "b_lambda": 0.5}, # Central Gaussian bump
    {"a_theta": 1e-3, "a_lambda": 20, "b_lambda": 0.8}, # High-energy focused
    {"a_theta": 1e-4, "a_lambda": 15, "b_lambda": 0.3}, # Low-energy focused
]

for params in example_params:
    W = parametric_weight(Y, X, **params)
    plot_weight(Y, X, W)
```





```
[51]: def evaluate_fom(a_theta, a_lambda, b_lambda, N=10000):
    """
    Evaluate the Figure of Merit (FOM) for a given set of Gaussian weight_
    ↪ parameters.

    Parameters
    -----
    a_theta : float
        Angular spread parameter controlling width in theta (Y).
    a_lambda : float
        Energy spread parameter controlling width in lambda (X).
    b_lambda : float
        Central energy location of the weighting.
    N : int, optional
        Number of simulated particles/events to use (default = 10000).

    Returns
    -----
    float
        Computed Figure of Merit (FOM) for the specified weight parameters.
    """
    X0 = data_all["With"][0]["X"]
    Y0 = data_all["With"][0]["Y"]
    #w_grid = np.exp(-a_theta * Y0**2) * np.exp(-a_lambda * (X0 - b_lambda)**2)
    #w_grid = a_theta*Y0**2 + a_lambda*(X0-b_lambda)
    w_grid = a_theta*Y0**2 + b_lambda*X0*(1+a_lambda*Y0**4)

    # Define a weight function that just returns the precomputed grid
    def weight_func(Y, X, run_id=None):
        return w_grid
```

```

    # Compute the FOM for this weighting (only using "With" EDM data)
    FOM = np.sum(compute_fom_over_lambda(data_all, N,
    ↪weight_func=weight_func, lambda_max=0.99))

    return FOM

example_params = [
    {"a_theta": 1e-4, "a_lambda": 1, "b_lambda": 0.5},
    {"a_theta": 1e-3, "a_lambda": 10, "b_lambda": 0.5},
    {"a_theta": 1e-3, "a_lambda": 20, "b_lambda": 0.8},
    {"a_theta": 1e-4, "a_lambda": 15, "b_lambda": 0.3},
]

'''for p in example_params:
    fom = evaluate_fom(**p)
    print(f"a={p['a_theta']:.1e}, a={p['a_lambda']:.1f}, b={p['b_lambda']:.
    ↪2f} → FOM={fom:.4f}")'''

```

```

[51]: 'for p in example_params:\n    fom = evaluate_fom(**p)\n
print(f"a={p['a_theta']:.1e}, a={p['a_lambda']:.1f},
b={p['b_lambda']:.2f} → FOM={fom:.4f}")'

```

```

[52]: def objective(trial):
    """
    Objective function for Optuna - maximise the Figure of Merit (FOM)
    by tuning the Gaussian weight parameters.
    """

    # --- Define parameter search ranges ---
    a_theta = trial.suggest_float("a_theta", -10, 10)
    a_lambda = trial.suggest_float("a_lambda", -10, 10.0)
    b_lambda = trial.suggest_float("b_lambda", -10, 10)

    # --- Evaluate FOM for these parameters ---
    fom = evaluate_fom(a_theta, a_lambda, b_lambda, N=10000)

    # Optuna minimises by default; we want to maximise, so return negative
    return -fom

# --- Create and run the study ---
study = optuna.create_study(direction="minimize", study_name="FOM optimisation")
study.optimize(objective, n_trials=10, show_progress_bar=True)

# --- Print best results ---
best_params = study.best_params
best_fom = -study.best_value

```



```

print("\nBest parameters found:")
for k, v in best_params.items():
    print(f"    {k} = {v:.6g}")
print(f"Maximum FOM = {best_fom:.4f}")

```

[I 2025-11-11 14:01:22,444] A new study created in memory with name: FOM optimisation

0%| | 0/10 [00:00<?, ?it/s]

[I 2025-11-11 14:01:25,880] Trial 0 finished with value: -206578.35911292103 and parameters: {'a_theta': -3.7535233192405197, 'a_lambda': 7.285146567266448, 'b_lambda': 7.671543725841417}. Best is trial 0 with value: -206578.35911292103.

[I 2025-11-11 14:01:29,211] Trial 1 finished with value: -206579.58858740993 and parameters: {'a_theta': -2.1558566551986296, 'a_lambda': 7.20630137225864, 'b_lambda': 4.8159112501369155}. Best is trial 1 with value: -206579.58858740993.

[I 2025-11-11 14:01:32,417] Trial 2 finished with value: -206610.31537625677 and parameters: {'a_theta': 5.021077500079269, 'a_lambda': -7.531157184219122, 'b_lambda': -9.443370533755548}. Best is trial 2 with value: -206610.31537625677.

[I 2025-11-11 14:01:35,694] Trial 3 finished with value: -206581.86543620762 and parameters: {'a_theta': -1.829916943680276, 'a_lambda': 7.8412508732230926, 'b_lambda': 4.448750364316139}. Best is trial 2 with value: -206610.31537625677.

[I 2025-11-11 14:01:38,680] Trial 4 finished with value: -206586.2536704933 and parameters: {'a_theta': -2.9670710942614287, 'a_lambda': 9.67264304298578, 'b_lambda': 9.082240838833407}. Best is trial 2 with value: -206610.31537625677.

[I 2025-11-11 14:01:41,829] Trial 5 finished with value: -206599.88622505526 and parameters: {'a_theta': -1.0552843224516657, 'a_lambda': 6.764729773993658, 'b_lambda': -7.246533248128573}. Best is trial 2 with value: -206610.31537625677.

[I 2025-11-11 14:01:44,699] Trial 6 finished with value: -206611.4123897411 and parameters: {'a_theta': 3.5051400149801957, 'a_lambda': 5.764841694603362, 'b_lambda': 8.863861878496145}. Best is trial 6 with value: -206611.4123897411.

[I 2025-11-11 14:01:47,843] Trial 7 finished with value: -206624.1981672291 and parameters: {'a_theta': 3.2689544982248826, 'a_lambda': 2.918194600309562, 'b_lambda': 9.526359664224803}. Best is trial 7 with value: -206624.1981672291.

[I 2025-11-11 14:01:50,812] Trial 8 finished with value: -206563.0361343366 and parameters: {'a_theta': 1.5100045989765896, 'a_lambda': -1.9422218318110787, 'b_lambda': 6.575578473187736}. Best is trial 7 with value: -206624.1981672291.

[I 2025-11-11 14:01:53,800] Trial 9 finished with value: -206618.00224888447 and parameters: {'a_theta': -2.708953527724532, 'a_lambda': 3.881178675198152, 'b_lambda': -7.432225597147819}. Best is trial 7 with value: -206624.1981672291.

Best parameters found:

```

a_theta = 3.26895
a_lambda = 2.91819
b_lambda = 9.52636

```

Maximum FOM = 206624.1982

```
[53]: optuna.visualization.plot_optimization_history(study)
```

```
[54]: # --- Extract the best parameters from Optuna ---
best_params = study.best_params
a_theta_best = best_params["a_theta"]
a_lambda_best = best_params["a_lambda"]
b_lambda_best = best_params["b_lambda"]

print("Best parameters:")
print(f"  a_theta = {a_theta_best:.3e}")
print(f"  a_lambda = {a_lambda_best:.3f}")
print(f"  b_lambda = {b_lambda_best:.3f}")

# --- Build the corresponding weight map ---
run_id = 7 # or whichever run you want to visualise
Y, X, Z = data_all["With"][run_id]["Y"], data_all["With"][run_id]["X"],
↳data_all["With"][run_id]["Z"]

W_opt = np.exp(-a_theta_best * Y**2) * np.exp(-a_lambda_best * (X -
↳b_lambda_best)**2)

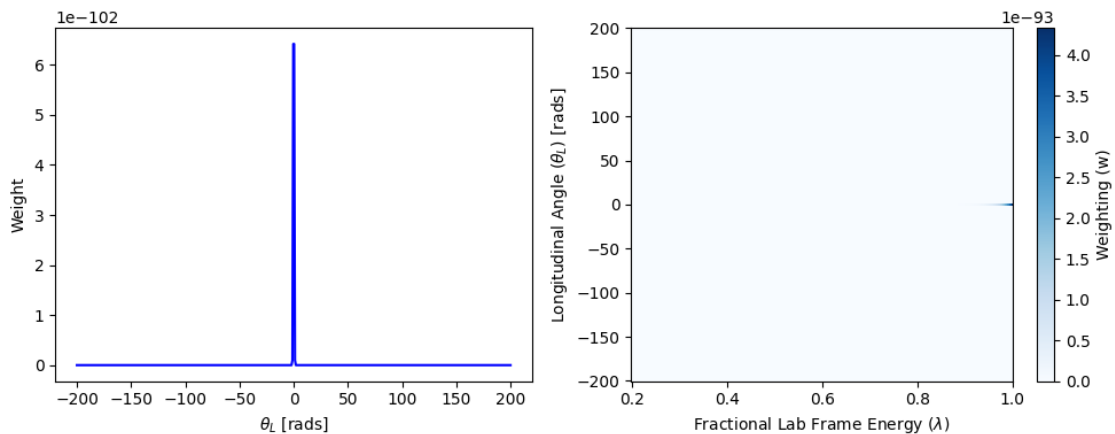
# --- Plot the 1D and 2D visualisations ---
plot_weight(
    Y, X, W_opt,
    name=f"Optimised Weight (a={a_theta_best:.1e}, a={a_lambda_best:.1f},
↳b={b_lambda_best:.2f})",
    #save_path="outputs/Weighting/Optimised_Weight.png"
)
```

Best parameters:

```

a_theta = 3.269e+00
a_lambda = 2.918
b_lambda = 9.526

```



```

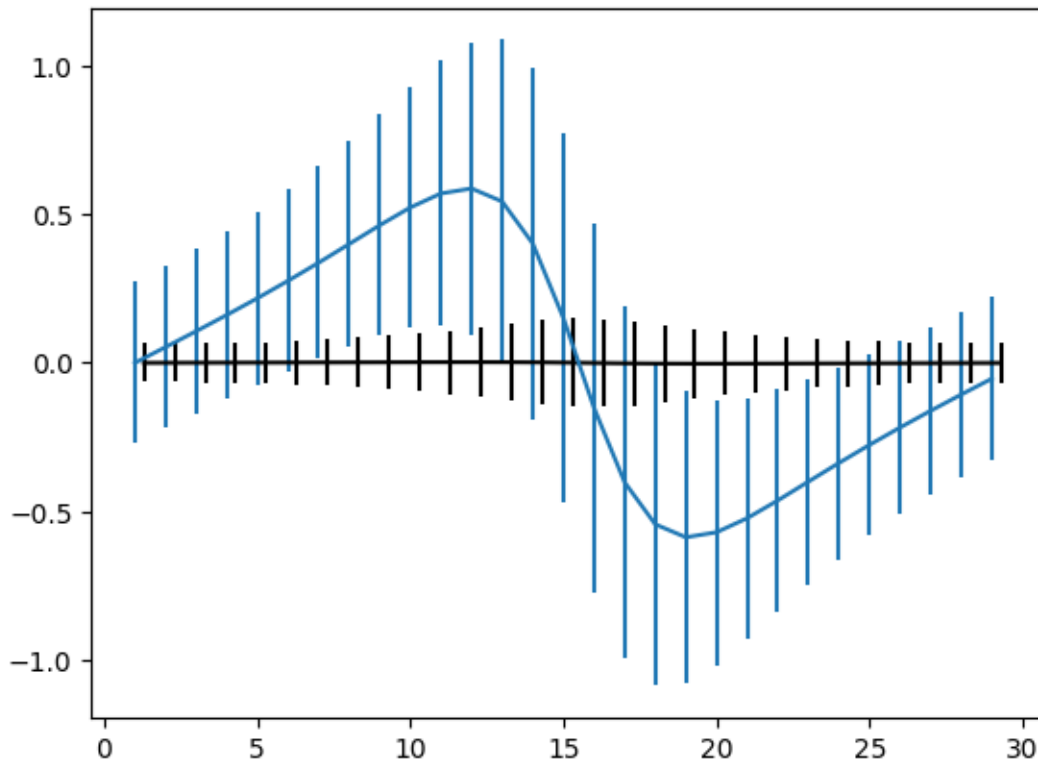
[55]: y = np.zeros(29)
y_uw = np.zeros(29)
yerr = np.zeros(29)
yerr_uw = np.zeros(29)

w1 = np.ones_like(W_opt)

l = 0.89
for i in range(29):
    grids = data_all[edm_state][i]
    lamb = grids['X'][0]
    lindex = np.argmin(np.abs(1-lamb))
    z = grids['Z'][:,lindex].transpose()
    #print(lamb, lindex)
    y[i], _, yerr[i] = calculate_metrics_array(grids['Z'][:,lindex],grids['Y'][:,lindex],1000,w=W_opt[:,lindex])
    y_uw[i], _, yerr_uw[i] = calculate_metrics_array(grids['Z'][:,lindex],grids['Y'][:,lindex],1000,w=w1[:,lindex])

plt.errorbar(np.arange(1.3,30.3, 1.0),y,yerr=yerr,color='black')
plt.errorbar(np.arange(1,30),y_uw,yerr=yerr_uw)
plt.show()

```



[]: